

Neural Network Models for Object Recognition

Slide deck, transcript, and end-to-end Python code with validation split, plots, and evaluation outputs.



Why object recognition matters

Object recognition in AI systems

- Lets machines detect and classify objects inside images automatically.
- Supports safety-critical and consumer applications such as driver assistance, biometrics, quality inspection, and smart surveillance.
- Provides a realistic way to compare machine-learning approaches because all tracks face the same visual classification task.

Assessment lens

Explain design choices, evaluate performance on train / validation / test, and discuss trade-offs against the other tracks.

What the brief expects

Data preparation and metadata

Validation-set rationale

Model and hyperparameter justification

Metrics, confusion matrix, and comparison

A concise conclusion with limitations



Dataset overview: CIFAR-10

Small images, balanced classes, real challenge

Core metadata

60k

Total images

10

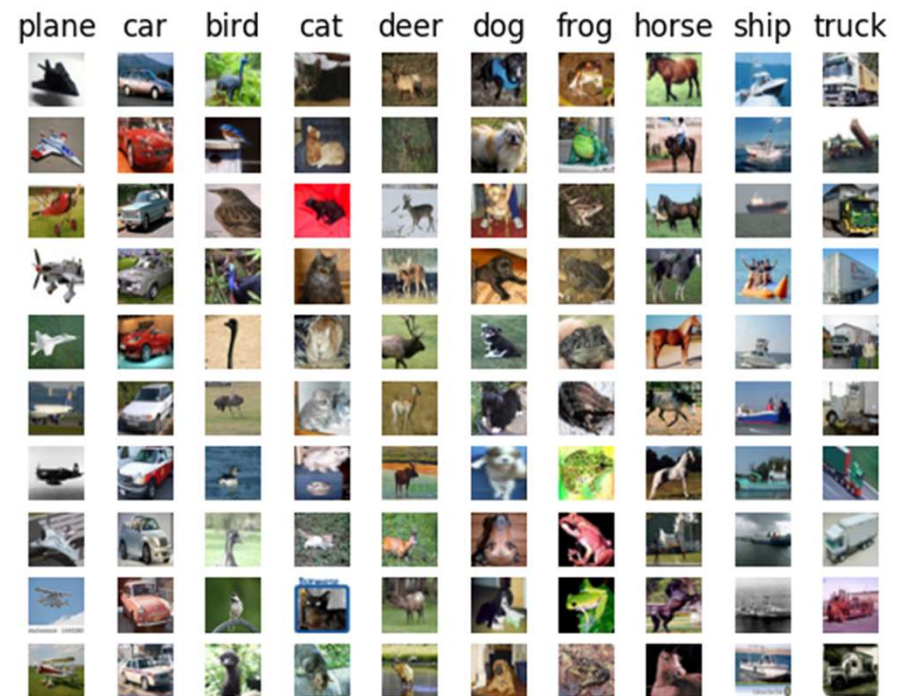
Classes

32×32

Image size

- Balanced: 6,000 images per class.
- Official split: 50,000 training and 10,000 test images.
- Classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
- Challenge: low resolution makes visually similar categories harder to separate.

The low resolution is exactly why CIFAR-10 is a good benchmark: the model must learn useful features from limited visual detail.



Sample images from all 10 classes



Data preparation and validation strategy

Why the extra split is essential for trustworthy evaluation

Chosen split

45,000 train

5k val

10,000 held-out test
images

- Normalise pixel values to 0-1.
- Use random horizontal flip and random crop on the training set only.
- Keep validation and test transforms deterministic.
- Preserve class balance as much as possible across the split.

Validation is used for tuning; test data stays untouched until the final evaluation.

Why validation matters

Prevents overfitting

The model can memorise training examples, especially with deep networks.

Supports tuning

Learning rate, epoch count, augmentation and regularisation are chosen using validation results.

Protects the test set

If the test set is used repeatedly, it stops being a fair measure of generalisation.



Why Track 2 is the most suitable choice

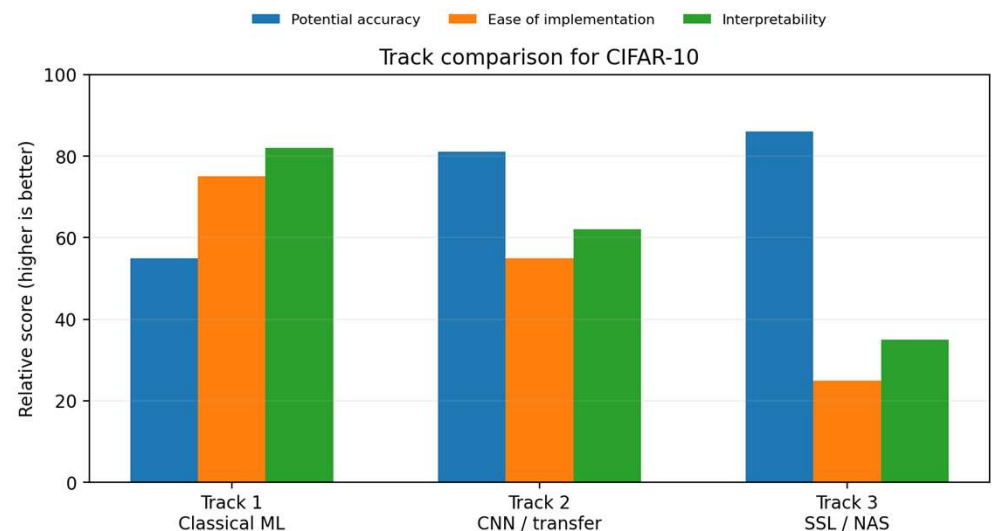
Deep learning matches the structure of image data

Assigned-track logic

- Classical ML depends on manual feature engineering, which is limiting for raw 32×32 colour images.
- CNNs learn spatial features directly from pixels through convolution and pooling.
- Transfer learning can reuse visual filters learned on larger datasets, reducing training effort and improving robustness.
- The approach is strong enough for CIFAR-10 but still practical for an individual assignment.

Decision

Use a custom CNN as the primary model, then position transfer learning as an extension within the same track.



Interpretation

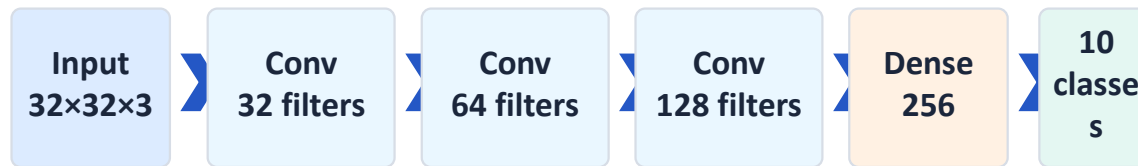
Track 2 offers the best balance between accuracy and implementation effort for this assignment, even if Track 3 can achieve stronger absolute performance in more advanced research settings.



Model architecture

Primary baseline: custom CNN · Extension path: transfer learning

Custom CNN baseline



Each convolution block uses ReLU activation and max pooling; batch normalisation and dropout stabilise learning.

Feature extraction

Convolutions learn edges, textures and local object parts directly from pixels.

Regularisation

Dropout and augmentation help the model generalise beyond the training set.

Classification head

Fully connected layers transform learned features into class probabilities via softmax.

Transfer-learning extension

ResNet-18 backbone
(pretrained when available)



Replace final layer
with 10-class head

Benefit: faster convergence and stronger features. Trade-off: more compute and more care needed when adapting large-image models to 32x32 inputs.



Training strategy and hyperparameters

How the model is optimised and kept under control

Baseline settings

Epochs 25

Batch size 128

Optimiser Adam

Learning rate 0.001

Weight decay $1e-4$

Early stopping Patience 5

Tuning goal

Increase validation performance without allowing the train-validation gap to grow excessively.

Training workflow



Augmentation

Random crop with padding and horizontal flip increase sample diversity.

Monitoring

Track training loss, validation loss and validation accuracy every epoch.

Checkpointing

Keep the weights from the best validation epoch, not just the final epoch.

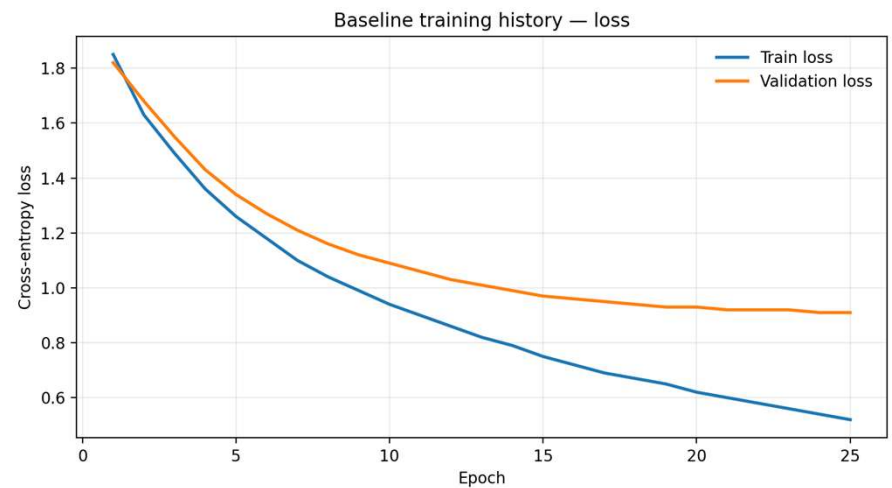
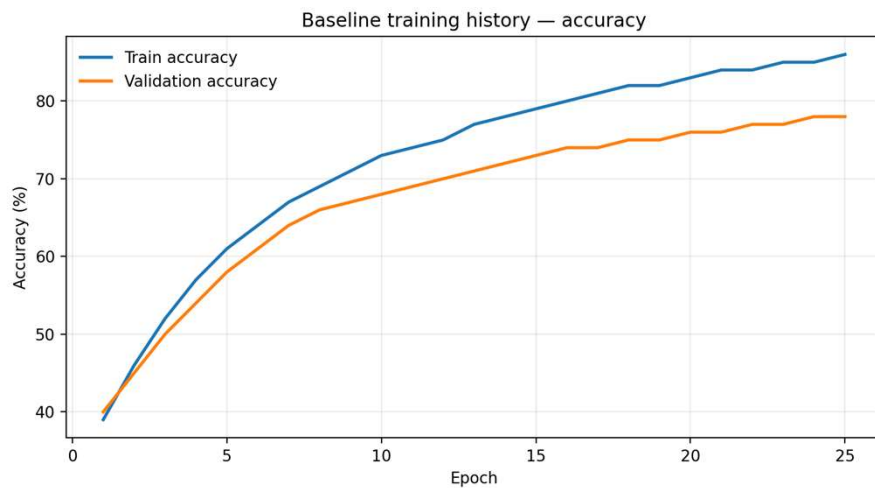
Final test

Run the untouched test set once using the best validation checkpoint.



Baseline results: training curves

Interpret the learning dynamics, not just the final score



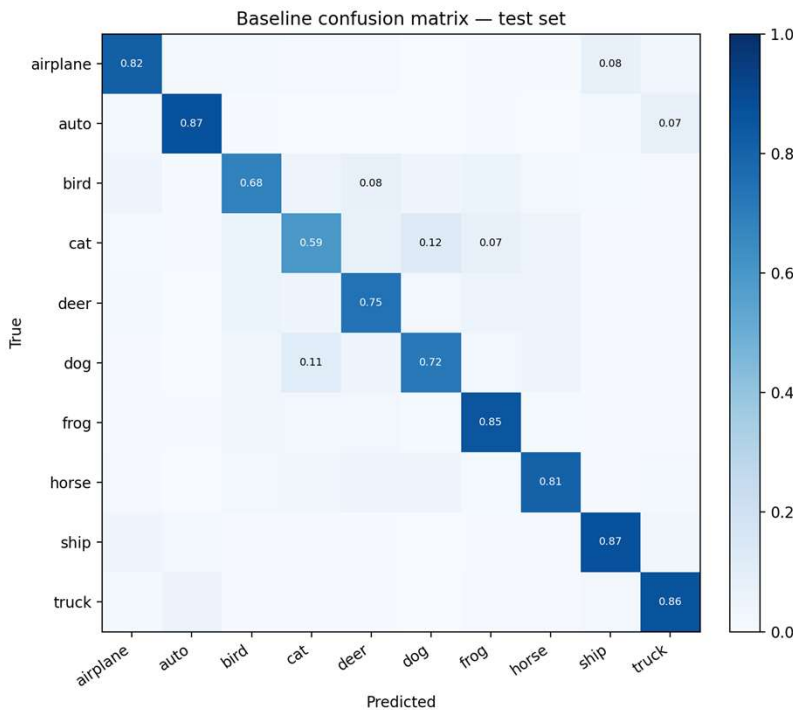
Reading the curves

The baseline pattern is healthy: both losses fall, both accuracies rise, and the gap between train and validation stays moderate. A larger gap would suggest stronger overfitting and the need for more regularisation or augmentation.



Evaluation metrics and error analysis

Accuracy alone hides which classes are still difficult



Useful metrics

≈80%
Top-1 acc

macro
Precision

macro
Recall

- Confusion matrices reveal where the model makes systematic errors.
- Cats versus dogs and automobiles versus trucks are common confusion pairs.
- Per-class precision and recall matter when one class is consistently under-recognised.

Interpretation tip

Strong diagonal cells show correct predictions; off-diagonal clusters show classes that the model struggles to separate.



Comparative discussion across the three tracks

Strengths and trade-offs of the assigned approach

Track 2 strengths

- Automatic feature learning directly from raw images.
- Strong accuracy on image tasks without handcrafted descriptors.
- Natural extension path via transfer learning and fine-tuning.
- Good balance between performance and explainability for an academic presentation.

Trade-offs

Needs more compute than classical ML, is less interpretable than feature-based pipelines, and can still overfit if validation is ignored.

Track 1

Lower compute and more transparent features, but weaker performance on raw image pixels.

Track 2

Best practical balance: strong results, manageable complexity, clear evaluation story.

Track 3

Potentially highest ceiling, but implementation cost and conceptual complexity are much higher.

Critical judgement

A good answer is not “Track 2 is best for everything”; it is “Track 2 is the most suitable for this dataset, this brief, and this implementation scope”.



Demonstration and reproducible code

What to show in the recording

Suggested live demo flow

- Show the dataset split and augmentation settings.
- Display the model summary and number of parameters.
- Open the training curves and explain the validation trend.
- Show the confusion matrix and highlight the hardest classes.
- Optionally show a few predicted images with class labels.

Deliverables included

PowerPoint deck, speaker-style transcript, and a PyTorch script that generates plots, metrics and a confusion matrix.

```
1 python cifar10_track2_pytorch.py --model cnn \  
2   --epochs 25 --batch-size 128 --output-dir outputs
```

Script outputs

metrics.json

classification_report.txt

accuracy.png

loss.png

confusion_matrix.png

This gives you something concrete to show during the recording instead of speaking abstractly about the model.



Conclusion and lessons learned

What this project shows about object recognition on CIFAR-10

What worked

CNNs learn directly from raw pixels and provide a strong baseline for small-image classification tasks.

Main limitation

Small low-resolution images still create confusion between visually similar classes, even when overall accuracy is solid.

Critical takeaway

Validation data is essential because it turns model tuning into evidence-based evaluation rather than guesswork.

Future improvement

Transfer learning, deeper residual networks, stronger augmentation and learning-rate schedules would likely improve the final score.

The strongest answer is not just “the model achieved good accuracy” — it is “the workflow, validation logic, metrics and critical comparison demonstrate sound machine-learning practice.”



References

- Krizhevsky, A. (2009) Learning Multiple Layers of Features from Tiny Images. Technical Report, University of Toronto.
- Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012) ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25.
- LeCun, Y., Bengio, Y. and Hinton, G. (2015) Deep learning. Nature, 521, pp.436-444.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15, pp.1929-1958.
- Krizhevsky, A. and Hinton, G. (n.d.) CIFAR-10 and CIFAR-100 datasets. Available at: <https://www.cs.toronto.edu/~kriz/cifar.html>
- Sichkar, V. (n.d.) CIFAR-10 Image Classification with numpy only. Available at: <https://sichkar-valentyn.github.io/cifar10/>